

LECTURE 8

Complexity Classes & Performance Tradeoffs

How growth classes guide real algorithm choices - without confusing asymptotic theory with actual runtime.

COMPARE

MEASURE

TRADE OFF

CHOOSE

Growth class → workload → practical cost → decision

L8

From “what grows faster?”
to “what should I use?”

What should you be able to do by the end?

Lecture 8 is about choosing algorithms using the complete workload - not just memorizing Big-O labels.

COMPARE

Compare common complexity classes using realistic input sizes.

INTERPRET

Explain why two algorithms with the same class can still run differently.

MEASURE

Describe how empirical timing can support theoretical analysis.

TRADE OFF

Reason about preprocessing, extra memory, and repeated queries.

CHOOSE

Justify which approach fits a given scenario.

Core idea: the “best” algorithm depends on input size, workload, assumptions, memory, and setup cost - not on one Big-O symbol alone.

Where Lecture 8 starts

Lecture 7 already taught growth rates. Today we use them to make decisions.

LECTURE 7

Count basic operations → simplify → classify the growth.

Examples: $\Theta(1)$, $\Theta(\log n)$, $\Theta(n)$, $\Theta(n \log n)$, $\Theta(n^2)$.

LECTURE 8

Given two valid choices, decide which is better for the actual workload.

Setup cost, repeated queries, memory, hardware, and input size matter.

DECISION

What should I actually use?

Today is not a new notation lecture. It is a “use the notation intelligently” lecture.

Same problem - three possible approaches

Suppose we repeatedly need to find a student registration number.

1. LINEAR SEARCH

Scan from the beginning until the target is found.

Setup: none

Lookup: $\Theta(n)$

Extra space: very low

2. SORT + BINARY SEARCH

Sort the records first, then repeatedly discard half the search range.

Setup: sorting

Lookup: $\Theta(\log n)$

Extra space: depends on sort

3. INDEX / HASH

Build an auxiliary structure for fast lookup.

Setup: build structure

Lookup: often very fast

Extra space: higher

The question is not “Which Big-O is smallest?” The question is “Which total cost is best for this workload?”

Case study: 1,024 student records

Start with the simplest question: we need to search once.

$n = 1,024$

**Find registration
number
FA23-BCS-115**



OPTION A

Search the unsorted list immediately using Linear Search.

OPTION B

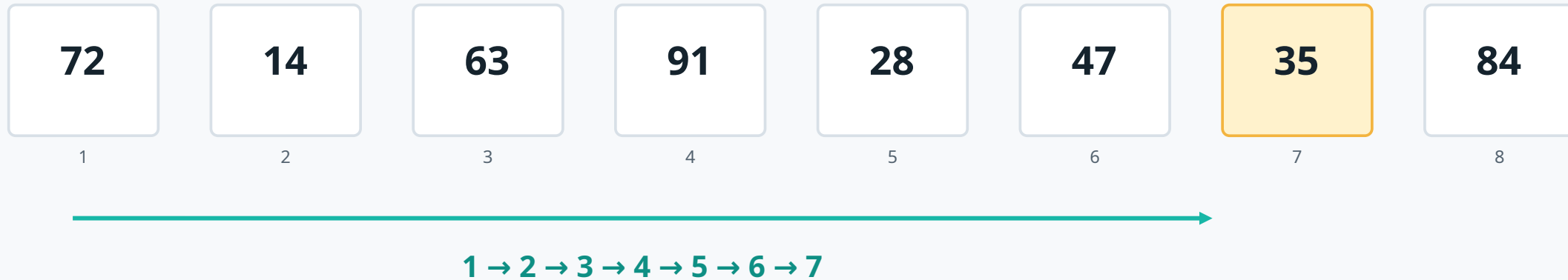
Sort the list first, then use Binary Search.

One search may favor a completely different choice than 10,000 searches.

Linear Search: look one item at a time

No preprocessing is needed, so we can start immediately.

Search for 35



Linear Search cost depends on where the target appears. In the worst case it may inspect all n items.

Linear Search: best, average, and worst case

This connects directly to the case analysis from Lecture 6.

BEST CASE

Target is the first item.

Comparisons = 1

Growth = $\Theta(1)$

AVERAGE SUCCESSFUL

Target position is spread across the list.

$\approx (n+1)/2$ comparisons

Growth = $\Theta(n)$

WORST CASE

Target is last or absent.

Comparisons = n

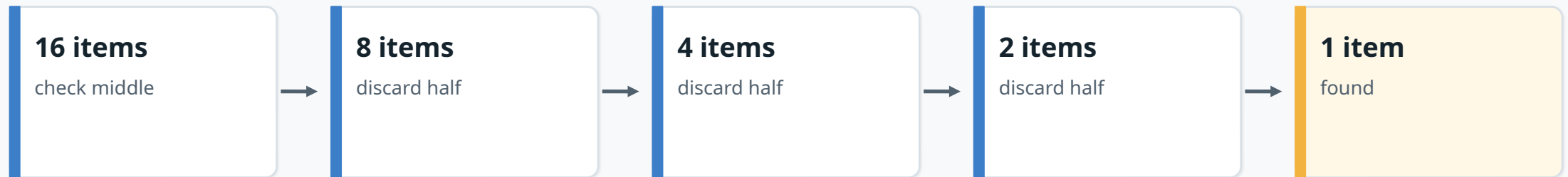
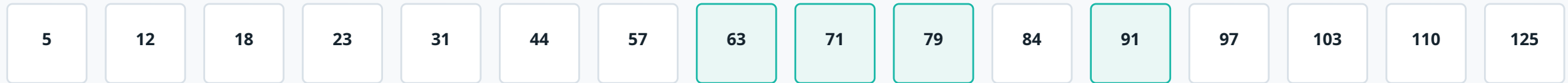
Growth = $\Theta(n)$

For repeated searches, the linear scan cost is paid again and again.

Binary Search: throw away half every step

Binary Search is fast because the remaining search range shrinks geometrically.

Search for 71



16 → 8 → 4 → 2 → 1. Repeated halving is why Binary Search has logarithmic growth.

For 1,024 items: 1,024 checks vs about 10

Now the asymptotic difference becomes concrete.

LINEAR SEARCH

Worst case: inspect every record.

1,024 comparisons

Growth: $\Theta(n)$

BINARY SEARCH

Repeatedly halve the remaining range.

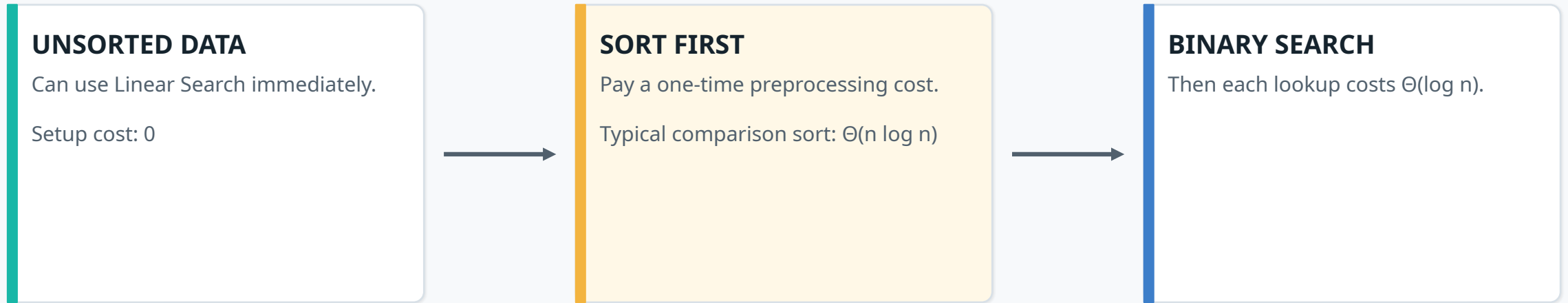
$\log_2(1,024) = 10$ comparisons approximately

Growth: $\Theta(\log n)$

$\approx 100\times$ fewer comparisons in this simplified comparison

But Binary Search has a requirement

The data must be sorted first. If it is not sorted, someone has to pay that setup cost.



A fast operation can still be a bad overall choice if its setup cost is larger than the work it saves.

One search: sorting first can be wasteful

Use the same $n = 1,024$ records.

OPTION A - LINEAR SEARCH

One worst-case scan:

1,024 comparisons

Total $\approx 1,024$

OPTION B - SORT + BINARY SEARCH

Approximate sort cost:

$$1,024 \times \log_2(1,024) = 10,240$$

Then one Binary Search ≈ 10

Total $\approx 10,250$

For a single search, Linear Search can be the better practical choice even though Binary Search has the better per-query growth.

Many searches change the decision

Let q be the number of searches on the same stable data.

Linear total $\approx qn$

Every search scans the list again.

Sort + Binary $\approx n \log n + q \log n$

Pay the setup cost once, then reuse it.

Repeated workloads can justify preprocessing because the setup cost is shared across many future operations.

Worked example: when does preprocessing pay off?

Again use $n = 1,024$, so $\log_2 n = 10$.

Linear: $1,024q$

Sort + Binary: $10,240 + 10q$

Find the point where sorting first becomes cheaper:

$$1,024q > 10,240 + 10q$$

$$1,014q > 10,240$$

$$q > 10.1$$

Around 11 or more searches: the preprocessing approach begins to win in this simplified model.

The break-even point is easier to see in a table

Same simplified model: $n = 1,024$.

q searches	Linear: $1,024q$	Sort + Binary: $10,240 + 10q$	Cheaper
1	1,024	10,250	Linear
5	5,120	10,290	Linear
10	10,240	10,340	Linear
11	11,264	10,350	Sort + Binary
100	102,400	11,240	Sort + Binary

The answer changes because q changes. Algorithm choice is a workload question.

Decision rule: compare total work, not one operation

A smaller per-query complexity does not automatically mean a smaller total cost.

1

How large is n ?

2

How many operations will happen?

3

Can I preprocess?

4

How often does the data change?

5

How much memory is available?

The Big-O label is evidence - not the entire decision.

Same Θ class does not mean same exact runtime

Asymptotic notation deliberately ignores constant factors.

$$C_1(n) = 10n$$

$$C_2(n) = 100n$$

At $n = 1,000$

ALGORITHM 1

$10 \times 1,000 = 10,000$ operations

ALGORITHM 2

$100 \times 1,000 = 100,000$ operations

Both are $\Theta(n)$, but one can still perform roughly ten times as much work.

Small n can hide the asymptotic advantage

A worse growth class can still be faster for small inputs if its constants are tiny.

$$A(n) = 100n$$

$$B(n) = n^2$$

n	100n	n ²	Faster
10	1,000	100	B
50	5,000	2,500	B
100	10,000	10,000	Tie
1,000	100,000	1,000,000	A

Asymptotic analysis describes long-run growth. It does not promise the winner at every small n.

Why theory and actual timing can differ

Growth class predicts scaling. Real machines add implementation details.

CONSTANT FACTORS

Some operations are simply more expensive than others.

HARDWARE

CPU speed, memory bandwidth, and storage matter.

LANGUAGE / COMPILER

Different implementations can add different overhead.

CACHE & MEMORY

Data locality can make the same operation much faster or slower.

INPUT CHARACTERISTICS

Best, average, and worst-case patterns can change observed time.

SYSTEM NOISE

Other processes, scheduling, and I/O can affect a timing run.

Empirical measurement: observe the implementation

Theoretical analysis and timing experiments answer related but different questions.

THEORETICAL ANALYSIS

Ask: how does the amount of work grow with n ?

- machine independent idea
- explains scalability
- gives asymptotic class

EMPIRICAL MEASUREMENT

Run the actual implementation and record elapsed time or other resource use.

- implementation specific
- includes constants and hardware
- validates practical behavior

Use both: theory explains the trend; experiments show what happens on the real system.

How to make a fair timing experiment

Bad experiments can produce misleading conclusions.

1

Use the same machine

2

Use comparable inputs

3

Repeat several trials

4

Use sufficiently large n

5

Report the setup clearly

If the experiment changes several things at once, you cannot tell which factor caused the timing difference.

Input choice can change the measured time

The same algorithm may behave differently on different input arrangements.

LINEAR SEARCH - TARGET FIRST

[35, 72, 14, 63, ...]

1 comparison

Very fast run

LINEAR SEARCH - TARGET MIDDLE

[..., 35, ...]

$\approx n/2$ comparisons

Typical successful run

LINEAR SEARCH - TARGET ABSENT

[72, 14, 63, ...]

n comparisons

Worst-case run

When measuring performance, state what kind of inputs were used.

Time-space tradeoff

Sometimes we deliberately use more memory so future operations take less time.

USE LESS SPACE

Store only the original data.

- little extra memory
- may rescan or recompute
- simpler structure

Possible cost: more time



USE MORE SPACE

Store an index, hash table, cache, or lookup table.

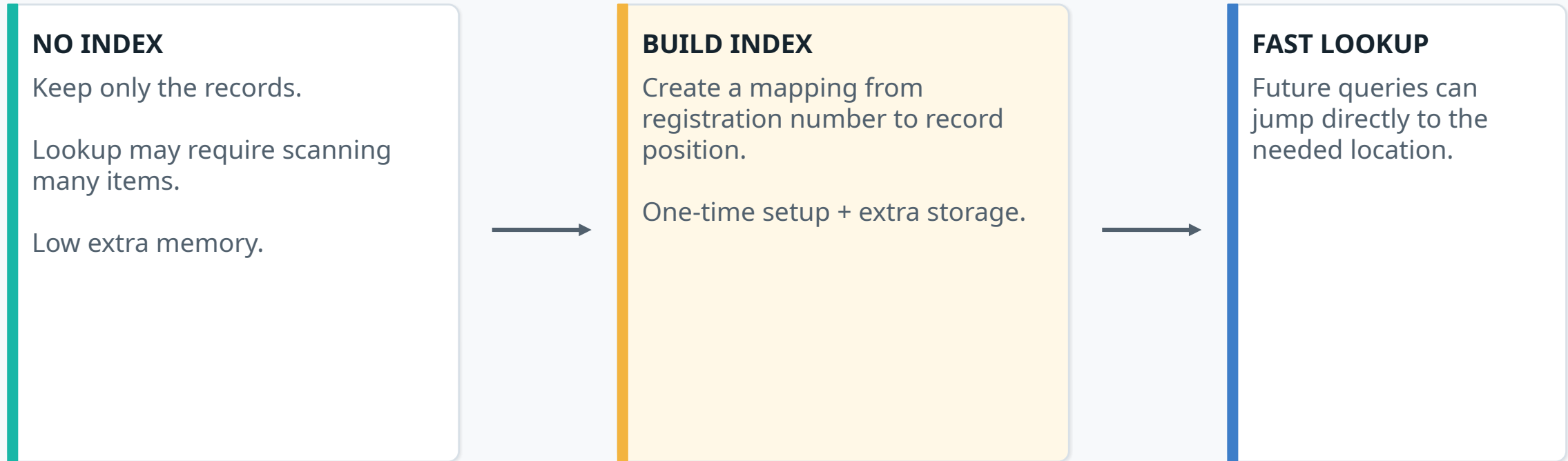
- more memory
- faster repeated access
- setup / maintenance cost

Possible benefit: less time

SPACE $\rightleftharpoons$ TIME

Example: build an index for repeated lookup

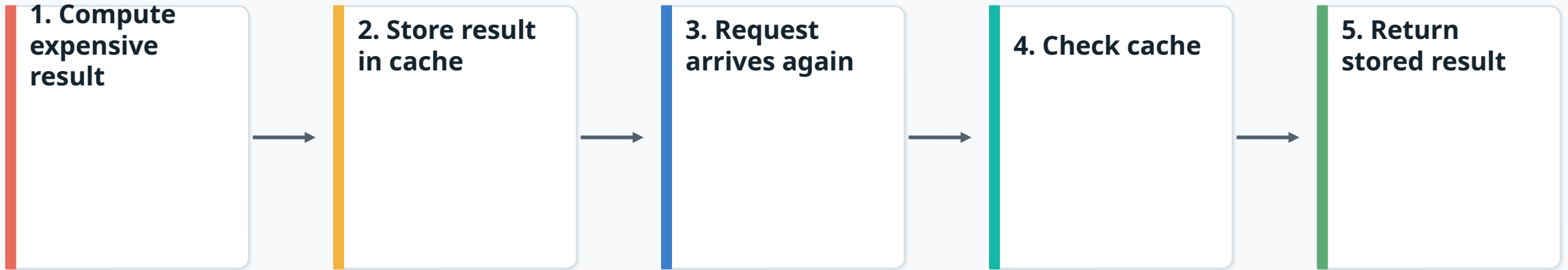
Extra structure can make searches much faster.



Indexes are attractive when lookups are frequent and the data is stable enough to justify maintaining the structure.

Example: caching avoids recomputation

Store a result once, reuse it later.



Caching uses extra memory to avoid repeating expensive work. The tradeoff is only worthwhile when reuse actually occurs.

The reverse tradeoff also exists

Sometimes memory is the scarce resource, so we accept more computation.

SAVE MEMORY

- use in-place algorithms
- avoid large lookup tables
- stream data instead of storing everything
- recompute values when needed

Result: less space, possibly more time

SAVE TIME

- cache results
- maintain indexes
- precompute useful values
- keep auxiliary structures

Result: more space, possibly less time

Algorithm design often balances resources rather than minimizing only one.

Second case study: detect duplicate registration numbers

Same problem, two correct approaches — one saves memory; the other saves a huge amount of time.

PROBLEM

Given n registration numbers, determine whether any registration number appears more than once.

EXAMPLE INPUT

$n = 10,000$

Think of 10,000 student records collected from multiple sources.

TWO IDEAS

A. Compare every pair.

B. Remember the IDs we have already seen.

Both algorithms can be correct. The interesting question is what each one costs.

Approach A — compare every pair

This uses almost no extra memory, but the number of comparisons grows quadratically.

ALGORITHM

```
for i ← 0 to n-2
  for j ← i+1 to n-1
    if A[i] = A[j]
      return DUPLICATE
return NONE
```

HOW MANY PAIRS?

Worst case: $n(n-1)/2$ comparisons

FOR $n = 10,000$

$10,000 \times 9,999 / 2 = 49,995,000$ pair comparisons

Time: $\Theta(n^2)$ • Extra space: $\Theta(1)$

Approach B — remember what we have already seen

Spend extra memory on a hash set so we do not compare each ID with every earlier ID.

ALGORITHM

```
SEEN ← empty set
for each id in A
    if id is in SEEN
        return DUPLICATE
    insert id into SEEN
return NONE
```

WHAT CHANGES?

Each ID is processed once instead of being compared with all later IDs.

FOR $n = 10,000$

About 10,000 membership checks + insertions, assuming average $O(1)$ hash operations.

Expected time: $\Theta(n)$ • Extra space: $\Theta(n)$

Same problem — different time-space tradeoff

The faster approach is not “free”; it buys speed by storing additional information.

COMPARE EVERY PAIR

Worst-case time
 $\Theta(n^2)$

Extra space
 $\Theta(1)$

Good when
 n is small or memory is
extremely tight

USE A HASH SET

Expected time
 $\Theta(n)$

Extra space
 $\Theta(n)$

Good when
 n is large and memory is
available

WHAT THE NUMBERS SAY

For $n = 10,000$:

Pairwise: 49,995,000 comparisons

Hash approach: about 10,000 lookups + insertions

The hash version uses much more memory, but can
avoid tens of millions of comparisons.

Lecture 8 lesson: choosing an algorithm means balancing growth, workload, and available resources.

Third case study: find the highest CGPA

A simple example of why solving more than the problem asks for can waste work.

PROBLEM

Given 10,000 student CGPAs, return only the highest CGPA.

IDEA A — SCAN

Keep the largest value seen so far.

One pass through the data.

IDEA B — SORT

Sort all 10,000 CGPAs, then take the last value.

Correct — but does much more work.

Both approaches are correct. The better choice depends on what output we actually need.

Scan once or sort everything?

Compare the amount of work for the same task.

SCAN FOR MAXIMUM

Start with $\text{max} \leftarrow A[0]$

Compare each remaining value with max

Update max when needed

Comparisons: $n-1$

For $n = 10,000$: 9,999 comparisons

Time: $\Theta(n)$

SORT, THEN TAKE MAX

Sort all n values first

Then read the last element

Typical comparison-sort work: about $n \log_2 n$

For $n = 10,000$: $\approx 132,877$ comparison-scale operations

Time: $\Theta(n \log n)$

Sorting solves a harder problem: it determines the complete order when we only asked for one maximum.

Lesson: do not solve a bigger problem than necessary

Asymptotic analysis helps reveal wasted work.

Find maximum $\rightarrow \Theta(n)$

CHOOSE THE SCAN

Need only the largest value, smallest value, or a simple aggregate? A direct pass may be enough.

Sort all values $\rightarrow \Theta(n \log n)$

CHOOSE SORTING

Need ranked output, median by position, repeated ordered access, or many later order-based operations? Sorting may be justified.

Lecture 8 lesson: compare algorithms against the exact problem requirement — not against a more difficult problem.

Fourth case study: repeated attendance-total queries

Preprocessing can turn repeated scanning into constant-time answers.

DATA

Daily attendance counts for a semester:

[8, 5, 6, 4, 7]

Question: "What was the total from day 2 through day 4?"

IDEA A — ADD EACH TIME

For every query, scan the requested days and add them.

Day 2 to 4:
 $5 + 6 + 4 = 15$

IDEA B — PRECOMPUTE

Build cumulative totals once.

Then answer every range with two stored values.

This is the same principle as indexing: pay a setup cost once, then reuse it many times.

Build a prefix-sum table once

Each stored value is the total from day 1 up to that position.

Daily attendance

8

5

6

4

7

Prefix totals

8

13

19

23

30

$$\text{Days 2-4} = \text{prefix}[4] - \text{prefix}[1] = 23 - 8 = 15$$

After preprocessing, a range total needs two lookups and one subtraction instead of scanning the whole range.

When does preprocessing pay off?

Compare one-time setup with the cost of many repeated queries.

NO PREPROCESSING

Each range query may scan up to n values.

One query: $\Theta(n)$ worst case

q queries: $\Theta(qn)$

Extra space: $\Theta(1)$

PREFIX SUMS

Build prefix table once: $\Theta(n)$

Each later query: $\Theta(1)$

q queries total: $\Theta(n + q)$

Extra space: $\Theta(n)$

FOR MANY QUERIES

Suppose $n = 1,000$ and $q = 10,000$.

Repeated scan can approach 10,000,000 additions.

Prefix method: about 1,000 setup operations + 10,000 constant-time queries.

We spend extra memory and preprocessing time because the setup cost is reused across many future queries.

A practical comparison table

Use this as a decision aid - not as an absolute rule.

Approach	Setup	Per lookup	Extra space	Good when...
Linear Search	none	$\Theta(n)$	very low	few searches / unsorted data
Binary Search	sorted data	$\Theta(\log n)$	low	many searches on stable sorted data
Index / Hash	build structure	often very fast	higher	very frequent lookup

Also ask: how often does the data change? Frequent updates can make preprocessing structures expensive to maintain.

Pause & predict — Case 1

Decide before the next slide reveals the worked answer.

One tiny search

- 20 values
- unsorted data
- only one search

YOUR DECISION

Would you scan immediately, or preprocess/sort first?

Give one reason — not just the Big-O label.

30-second class discussion → commit to an answer before we reveal it.

Reveal — Case 1: a tiny list, one search

Choose the simplest approach that meets the workload.

$n = 20$

WORKLOAD

One search only. Data is unsorted.

OPTION A

Linear Search

No setup.
At most 20 comparisons.

OPTION B

Sort first + Binary Search

Sorting overhead is larger than the work saved for a single tiny search.

Decision: use Linear Search.

Pause & predict — Case 2

Decide before the next slide reveals the worked answer.

Many repeated searches

- 1,000,000 records
- 50,000 lookups
- data changes rarely

YOUR DECISION

Would a one-time preprocessing/indexing cost make sense now?

Give one reason — not just the Big-O label.

30-second class discussion → commit to an answer before we reveal it.

Reveal — Case 2: one million stable records, 50,000 searches

Now the workload is completely different.

WORKLOAD

$n = 1,000,000$

50,000 lookups

Data changes rarely

Memory is available

LINEAR SCAN

Repeated $\Theta(n)$ scans would repeat enormous work across 50,000 queries.

PREPROCESS / INDEX

A one-time setup cost can be reused thousands of times. Fast repeated lookup becomes very attractive.

Decision: preprocessing or indexing is likely worthwhile.

Pause & predict — Case 3

Decide before the next slide reveals the worked answer.

Memory is the constraint

- limited RAM
- moderate data size
- only occasional searches

YOUR DECISION

Does the approach with the fastest lookup still automatically win?

Give one reason — not just the Big-O label.

30-second class discussion → commit to an answer before we reveal it.

Reveal — Case 3: memory is extremely limited

The asymptotically fastest lookup may not fit the machine.

CONSTRAINTS

- limited RAM
- moderate data size
- only occasional searches
- simple embedded device

LARGE INDEX

Fast lookup, but the auxiliary structure may consume unacceptable memory.

LOW-SPACE APPROACH

A scan or compact sorted representation may be slower per lookup but fit the resource budget.

Decision: choose the algorithm that satisfies the real resource constraints, not merely the smallest Big-O lookup time.

Three common mistakes

These are exactly the traps Lecture 8 is meant to prevent.

MISTAKE 1

"Binary Search is always better than Linear Search."

No - Binary Search needs sorted data and setup may not be worth it.

MISTAKE 2

"Same Θ class means same speed."

No - constants, implementation, and hardware still affect actual runtime.

MISTAKE 3

"The smallest time complexity is automatically best."

No - memory, preprocessing, updates, and input size also matter.

Checkpoint: choose the better approach

Do not answer from Big-O alone. Use the workload described.

A

20 unsorted values
One search

Linear Search or preprocess first?

Reason: _____

B

1,000,000 stable records
50,000 searches

Would preprocessing/indexing make sense?

Reason: _____

C

Memory extremely limited
Few searches

Would a large lookup table always be a good tradeoff?

Reason: _____

Checkpoint answers

The reason matters more than the label.

A - ONE SMALL SEARCH

Linear Search is sensible.
Preprocessing overhead may cost more than the single scan it replaces.

B - MANY REPEATED SEARCHES

Preprocessing/indexing is likely worthwhile because the setup cost is reused across thousands of queries.

C - MEMORY CONSTRAINED

Not necessarily. Faster lookup can require extra storage; the memory cost may make the tradeoff unacceptable.

A five-question checklist for algorithm choice

Run through this before declaring one algorithm “better.”

1

How large can n become?

2

One operation or many repeated operations?

3

Is preprocessing allowed?

4

How much memory is available?

5

How often does the data change?

Lecture 8 - the whole idea in one sentence

Use asymptotic growth to understand scaling, then choose using the complete practical workload.



Do not choose an algorithm from one Big-O label alone.

Next: Recursive Analysis & Substitution Method

Lecture 9 starts a new story: how do we describe the cost of an algorithm that calls itself?

TODAY - L8

Choose between alternative algorithms using complexity, workload, preprocessing, memory, and measurements.



NEXT - L9

Write recursive running times as recurrence relations and solve them using substitution.



AFTER THAT

Recurrence Trees → Master Theorem

Recommended reading for this lecture: Levitin Ch. 2; CLRS Ch. 3.